IT4931 / IT5384 / IT4043E · SOICT · HUST

Data Lakehouse

Lecture 4 — open table formats

Iceberg · Delta · Hudi · object storage

Learning outcomes

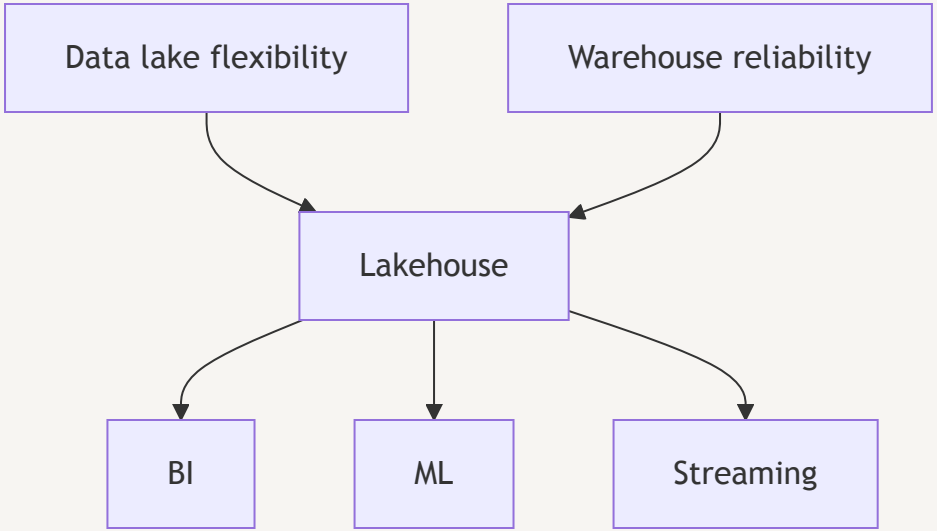
1. **Explain** lakehouse as lake flexibility + warehouse reliability
2. **Contrast** warehouse · lake · lakehouse stacks
3. **Describe** open table formats (metadata, ACID, time travel)
4. **Outline** Iceberg strengths; compare Delta & Hudi briefly

Roadmap today

1. **4.1** Motivation — why lakehouse
2. **4.2** Lakehouse & open table formats
3. **4.3** Iceberg in depth
4. **4.4** Delta · Hudi · medallion

4.1 Motivation

Why lakehouse in 2026?



- Platforms run on **object storage** (S3 / ADLS / GCS) + **table formats**
- One physical layer for **BI + ML + streaming** with transactions & schema mgmt

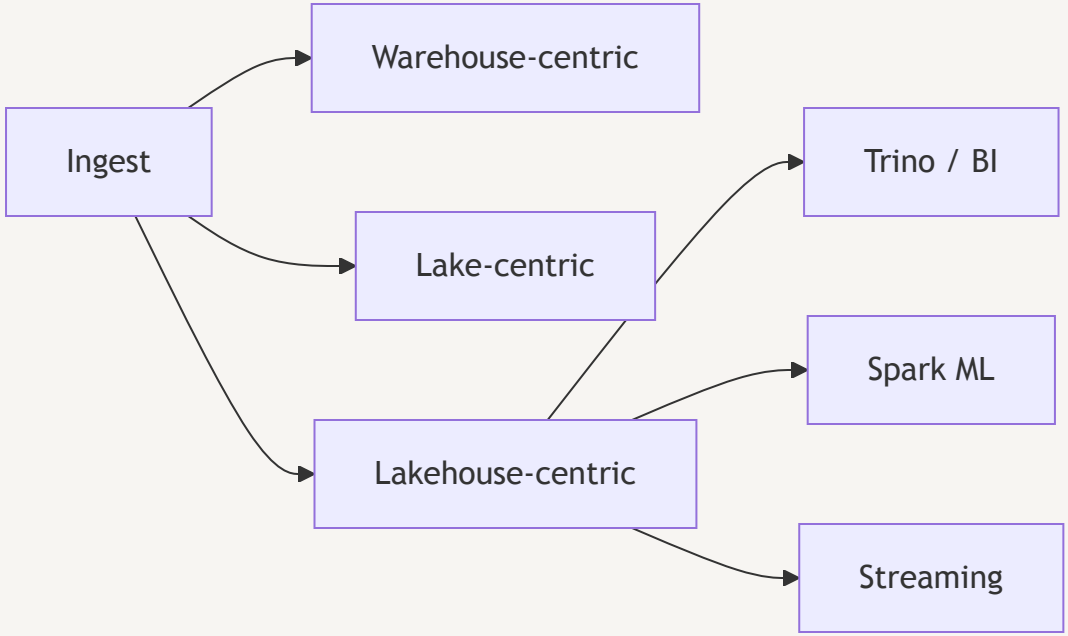
Warehouse vs lake vs lakehouse

Aspect	Warehouse	Lake	Lakehouse
Data	Cleaned structured	Mostly raw / all types	Raw + refined
Schema	On-write	On-read	Both + ACID tables
Use	BI / reporting	ML, logs, explore	BI + ML + stream
Storage	Engine-coupled	Object files	Object + **table format**

Mental model

Metaphor	Meaning
Warehouse	“Hotel kitchen” — cleaned ingredients, stable menu (BI)
Lake	Cheap store of *everything* — flexible, risk of swamp
Lakehouse	Same cheap storage + **proper ACID tables** for many engines

Typical stacks



- **Warehouse-centric** — BI first
- **Lake-centric** — dump files; governance optional
- **Lakehouse-centric** — Iceberg/Delta/Hudi tables shared by engines

Coexistence in real orgs

- Warehouse for **finance / critical BI**
- Lake/lakehouse for **events, ML, features**
- Kafka / Flink / Spark SS move data between them

Platform skill: place each workload correctly.

4.2 Lakehouse & open table formats

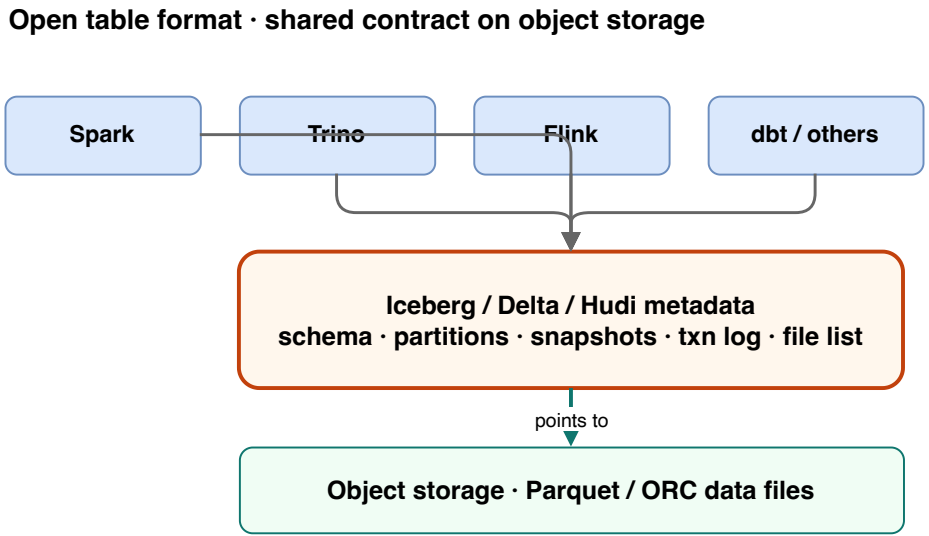
What is a lakehouse?

Object storage + database-like management:

- **ACID** on tables
- Schema **enforcement / evolution**
- **Time travel** / snapshots
- **Multi-engine** access (Spark, Trino, Flink, …)

Mechanism: **open table formats** over Parquet/ORC files.

Open table formats — concept



Engines share one table · ACID / time travel / evolution live in metadata

Adds schema · partitions · file inventory · snapshots · MERGE/UPDATE/DELETE — prune via metadata; multi-engine share.

4.3 Iceberg in depth

Apache Iceberg — overview

- Open format for large analytic tables on lakes
- **Engine-agnostic** (Spark, Trino, Flink, Hive, …)
- **Snapshots + manifests** track current data files
- Data stays as files; Iceberg is the **metadata contract**

Hidden partitioning

Classic dirs	Iceberg
`/dt=…/country=…/` must match filters	Query by **columns**; planner maps to partitions
Changing layout breaks paths	**Partition evolution** without rewriting old data

Schema & partition evolution

- **Schema:** add/drop/rename/reorder; stable **column IDs**
- **Partition spec:** change over time; old + new layouts coexist
- Metadata-only where possible → safer pipelines

Snapshots & time travel

- **Snapshot** = table state at a commit
- Query **as-of** timestamp / snapshot ID
- **Rollback** to a previous snapshot
- Audit, debug, reproduce past results

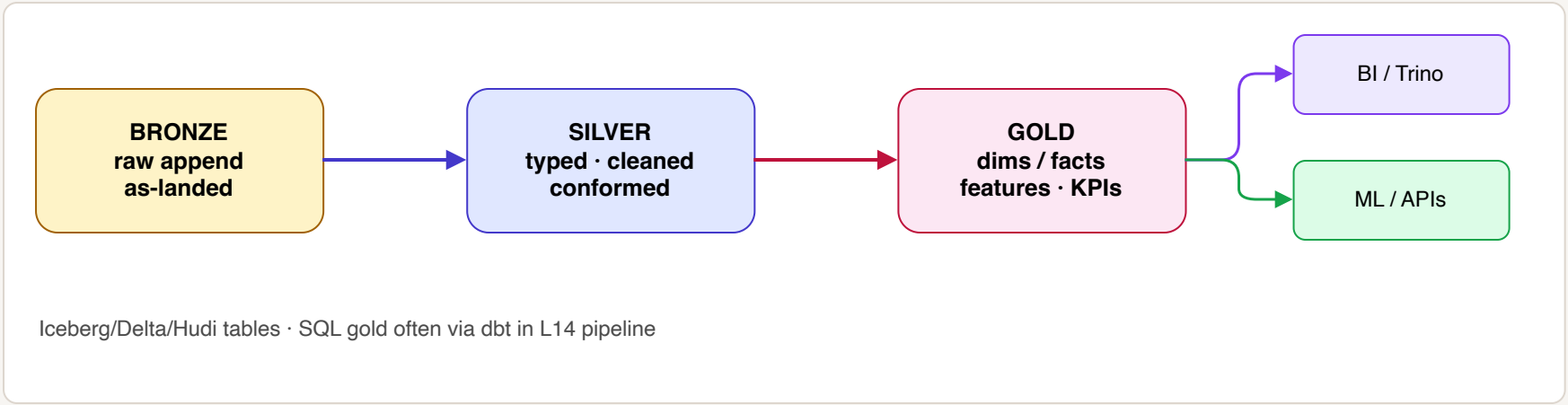
4.4 Delta · Hudi · medallion

Iceberg vs Delta vs Hudi

Format	Strength (high level)
Iceberg	Engine-agnostic analytics; strong metadata / hidden partitions
Delta Lake	Spark/Databricks depth; medallion batch patterns
Hudi	CDC / upsert / incremental (CoW / MoR)

All: ACID · schema evolution · time travel · multi-engine (to varying degrees).

Medallion sketch (bronze → gold)



Same Iceberg/Delta/Hudi tables · SQL gold often via **dbt** in the **L14** pipeline (not a separate dbt-only lecture).

Course links & projects

Thread	Lectures
Storage path	L03 HDFS → **L04** object + OTF
Query	L07 Trino on lake tables
Ingest / stream	L08 Kafka · L11–L12 Spark SS / Flink
Architecture	L09 Lambda/Kappa → lakehouse + streams
Capstone	**L14** pipeline (Iceberg · dbt · GE · Airflow)

Projects: Iceberg / Delta labs or format comparison papers welcome.

Takeaways

1. Lakehouse = **open tables on object storage**
2. Formats add **ACID, evolution, time travel, multi-engine**
3. Iceberg / Delta / Hudi — pick by ecosystem & upsert needs
4. Coexists with warehouses; unifies Spark/Trino/stream readers

Next

L05 — NoSQL